

UNIVERSIDAD TECNOLÓGICA NACIONAL

*Tecnicatura Universitaria en
Programación*

Asignatura: Programación II

**Trabajo Práctico Integrador: Food Store - Sistema
de Gestión de Pedidos**

Alumno: Franco Stefano Ghiossi Hrebin

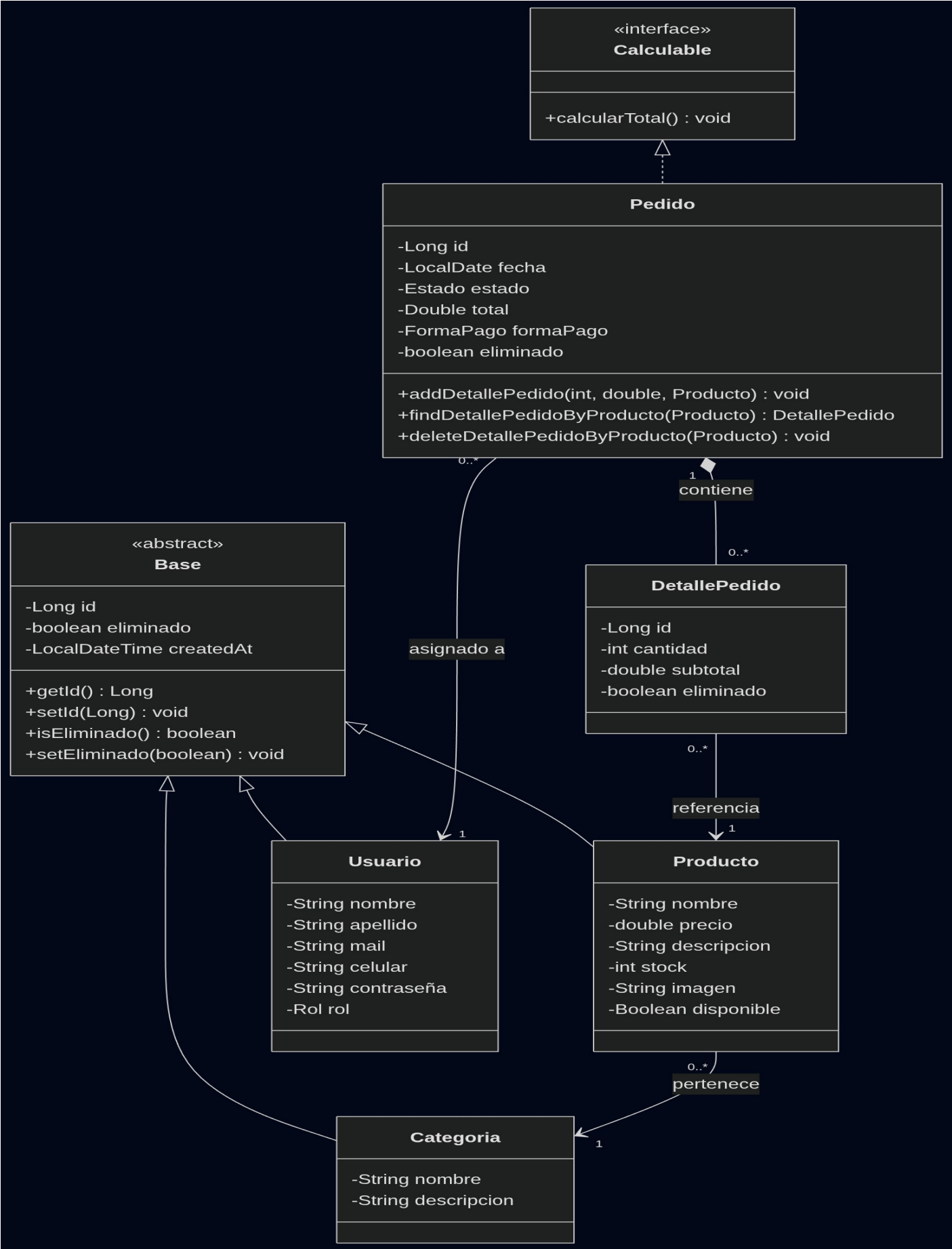
1. Introducción

El presente informe documenta el desarrollo de "Food Store", un sistema de gestión de pedidos en consola diseñado bajo el paradigma de Programación Orientada a Objetos (POO) en Java. La aplicación resuelve las problemáticas habituales de un negocio gastronómico, permitiendo administrar de forma integral categorías, productos, usuarios y pedidos. Al no contar con persistencia en bases de datos relacionales en esta etapa, toda la lógica y consistencia de los datos se sostiene en memoria mediante el uso estratégico de colecciones de Java.

2. Marco Teórico y Decisiones de Diseño

- **Arquitectura y Responsabilidades:** Se optó por una separación clara de responsabilidades estructurada en tres paquetes principales: *entities* (modelado de datos y encapsulamiento), *service* (*DataStoreService* como el motor central que nuclea las colecciones y la lógica de negocio) y *main* (*Main* como la interfaz gráfica por consola que interactúa con el usuario). Esto garantiza que la vista no manipule directamente los datos, promoviendo el bajo acoplamiento.
- **Abstracción y Herencia:** Para evitar la duplicación de código redundante (atributos como *id*, *eliminado* o *marcas de tiempo*), se definió la clase abstracta *Base*. Entidades como *Usuario*, *Producto* y *Categoría* heredan estas características, unificando el comportamiento estructural del modelo.
- **Contratos e Interfaces:** La interfaz *Calculable* provee un contrato estricto mediante el método *calcularTotal()*. La entidad *Pedido* implementa dicha interfaz de forma obligatoria, aislando la lógica aritmética del cálculo de totales de los flujos de control del menú.
- **Manejo de Excepciones y Robustez:** El sistema rechaza el uso de excepciones genéricas para el control de errores de negocio. Se implementaron excepciones personalizadas (*NegocioException*, *StockInvalidoException* y *EntidadNoEncontradaException*) que permiten interrumpir los flujos inválidos de forma controlada y notificar de manera precisa al usuario a través de la interfaz.

3. Diagrama de Clases (UML)



4. Gestión de Consistencia y Rollback en Memoria

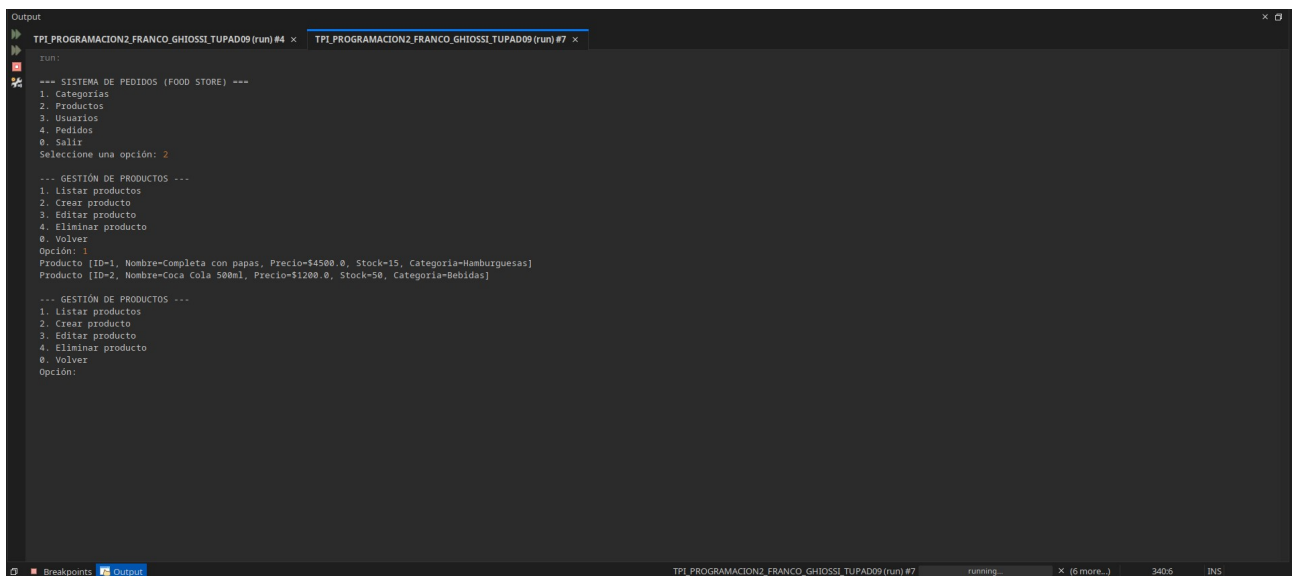
Uno de los mayores desafíos del proyecto radica en simular el comportamiento transaccional de una base de datos utilizando únicamente colecciones (List).

En el método registrarPedido, el operador puede solicitar múltiples productos. Si un producto de la lista cuenta con stock insuficiente, el sistema no debe dejar el pedido "a medias" ni descontar stock de los productos procesados previamente en ese mismo intento. Para resolver esto, el servicio implementa una política de **Rollback Manual**: antes de modificar los objetos en memoria, se resguardan los estados previos de los stocks afectados. Si salta una excepción en medio del bucle, el bloque catch intercepta el error, restaura los valores originales y propaga la excepción hacia la consola.

5. Sección de Evidencias

Captura 1: Listado Inicial de Productos

Al ingresar a la sección de productos y listar las opciones disponibles, se puede observar la precarga exitosa del sistema en memoria. El producto "Completa con papas" inicia formalmente con un stock de 15 unidades.



```
Output
TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #4 x TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #7 x
run:
--- SISTEMA DE PEDIDOS (FOOD STORE) ---
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 2

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción: 1
Producto [ID=1, Nombre=Completa con papas, Precio=$4500.0, Stock=15, Categoría=Hamburguesas]
Producto [ID=2, Nombre=Coca Cola 500ml, Precio=$1200.0, Stock=50, Categoría=Bebidas]

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción:
```

Captura 2: Apertura y Configuración de un Pedido

Se inicia la creación de un nuevo pedido asignado al Usuario con ID 1 y seleccionando la opción de pago en "Efectivo". El sistema queda a la espera de la carga de ítems.

```
Output
TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #4 x TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #7 x
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 2

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción: 1
Producto [ID=1, Nombre=Completa con papas, Precio=$4500.0, Stock=15, Categoría=Hamburguesas]
Producto [ID=2, Nombre=Coca Cola 500ml, Precio=$1200.0, Stock=50, Categoría=Bebidas]

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción: 0

--- SISTEMA DE PEDIDOS (FOOD STORE) ---
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 4

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción: 2
Ingrese ID del Usuario: 1
Seleccione Forma de Pago (1. TARJETA, 2. TRANSFERENCIA, 3. EFECTIVO):
3
Ingrese ID del Producto a comprar:
```

Captura 3: Carga de Ítems Exitosa y Cálculo del Total

Se agregan 2 unidades del producto ID 1. Al finalizar el pedido, el sistema ejecuta de forma transparente el método `calcularTotal()`, arrojando un importe exacto de \$9000.0 y confirmando la actualización del stock.

```
Output
TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #4 x TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #7 x
Opción: 1
Producto [ID=1, Nombre=Completa con papas, Precio=$4500.0, Stock=15, Categoría=Hamburguesas]
Producto [ID=2, Nombre=Coca Cola 500ml, Precio=$1200.0, Stock=50, Categoría=Bebidas]

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción: 0

--- SISTEMA DE PEDIDOS (FOOD STORE) ---
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 4

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción: 2
Ingrese ID del Usuario: 1
Seleccione Forma de Pago (1. TARJETA, 2. TRANSFERENCIA, 3. EFECTIVO):
3
Ingrese ID del Producto a comprar: 1
Cantidad: 2
¿Desea agregar otro producto a este pedido? (S/N): n
|Pedido registrado de forma segura y stock actualizado!

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción: 1
```

Captura 4: Control de Excepciones (Intento de sobrecompra)

Se abre un nuevo pedido para simular un caso crítico de negocio. Se intenta solicitar una cantidad de 999 unidades del producto ID 1 (cuyo stock real remanente es de 13). El sistema interrumpe el flujo, deniega la operación y lanza de manera controlada la excepción por pantalla sin romper la ejecución de la app.

```
Output
TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #4 x TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #7 x
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 4

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción: 2
Ingrese ID del Usuario: 1
Seleccione Forma de Pago (1. TARJETA, 2. TRANSFERENCIA, 3. EFECTIVO):
3
Ingrese ID del Producto a comprar: 1
Cantidad: 2
¿Desea agregar otro producto a este pedido? (S/N): n
¡Pedido registrado de forma segura y stock actualizado!

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción: 2
Ingrese ID del Usuario: 1
Seleccione Forma de Pago (1. TARJETA, 2. TRANSFERENCIA, 3. EFECTIVO):
3
Ingrese ID del Producto a comprar: 1
Cantidad: 999
¿Desea agregar otro producto a este pedido? (S/N): n
ERROR AL PROCESAR PEDIDO: Stock insuficiente para el producto: Completa con papas (Disponibles: 13)

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción:
```

Captura 5: Verificación de Consistencia (Efecto Rollback)

Para constatar que el error del paso anterior no corrompió los datos en memoria, se regresa al menú de productos y se listan nuevamente. Se evidencia que el stock de la "Completa con papas" permaneció intacto en 13 unidades, validando el correcto funcionamiento del rollback lógico desarrollado.

```
Output
TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #4 x TPI_PROGRAMACION2_FRANCO_GHIOSSI_TUPAD09 (run) #7 x
Opción: 2
Ingrese ID del Usuario: 1
Seleccione Forma de Pago (1. TARJETA, 2. TRANSFERENCIA, 3. EFECTIVO):
3
Ingrese ID del Producto a comprar: 1
Cantidad: 999
¿Desea agregar otro producto a este pedido? (S/N): n
ERROR AL PROCESAR PEDIDO: Stock insuficiente para el producto: Completa con papas (Disponibles: 13)

--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Cambiar estado / forma de pago
4. Eliminar pedido (Baja lógica)
0. Volver
Opción: 0

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 2

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción: 1
Producto [ID=1, Nombre=Completa con papas, Precio=$4500.0, Stock=13, Categoría=Hamburguesas]
Producto [ID=2, Nombre=Coca Cola 500ml, Precio=$1200.0, Stock=50, Categoría=Bebidas]

--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver
Opción: 1
```

6. Conclusión

El desarrollo de este integrador permitió afianzar los conceptos fundamentales de la programación orientada a objetos en un entorno puramente lógico y de memoria. El diseño modular aplicado no solo asegura que el código sea mantenible y prolijo bajo las pautas evaluativas de la cátedra, sino que deja el terreno perfectamente preparado para una futura migración hacia persistencia real mediante JDBC sin alterar la estructura central del negocio.